

TEMA 2 – Objetos del Usuario II

Objetivo del tema

En este tema aprenderemos sobre los Objetos definidos por el usuario

2.1 Encapsulación

Consulta la presentación **UD04 - Objetos del Usuario 2**

La **Programación Orientada a Objetos** es un paradigma de programación que organiza el diseño de software en torno a objetos. Hasta aquí, nada que no sabías.

Una de las características más importantes de la POO es la **Encapsulación**. La idea tras la encapsulación es ocultar los detalles internos de una clase. De esta forma, alguien que quiera usar la clase solamente tiene que conocer cómo usarla, es decir, sus métodos públicos.

Por ejemplo, la clase Coche de Carreras tiene cincuenta métodos y veinte atributos. Para evitar errores, inconsistencias, etc. optamos por encapsular el código, dejando únicamente como públicos a los métodos que necesita quien use esta clase: conducir (), acelerar (), getMotor (), etc.

La encapsulación NO se hace solo a nivel de clase. Se puede hacer a nivel de paquete, o a nivel de método.

Vale, esto ya te suena de antes.

Aunque todo esto que has leído está extraído de unos apuntes de Java, esto también se aplica en JavaScript. No queremos que ciertas partes de un objeto se toquen. La forma más directa es haciendo un método o atributo privado mediante #.

2.2 Módulos

Definimos **módulo** como un archivo JavaScript que se puede exportar o importar para añadir funcionalidades a nuestras páginas web. Es un concepto similar a las librerías de Java (salvando las distancias). Separar el código en módulos permite encapsular el código de forma adecuada, mejora el mantenimiento y la reutilización de componentes.

Cada **módulo** en JavaScript tiene su propio **ámbito local (module scope)**: lo que declaras dentro de un fichero .js **NO** es visible en otros ficheros a menos que lo **exportes** e **importes** explícitamente.

Podemos añadir un módulo a una página indicando:

```
<script type="module"src="./ruta.js"></script>
```

Exportar / Importar

Supongamos que tenemos este fichero JavaScript **mathUtils.js** y queremos usar sus métodos etc. en otro fichero JavaScript.

```
const PI = 3.14159;

function sumar(a, b) {
  return a + b;
}

class Calculadora { ... }
```

Ni PI, ni sumar () ni la clase Calculadora son accesibles desde otro JavaScript. Para eso, tenemos que marcarlos con **export**. Esto es lo que se llama **Exportación Nombrada**.

```
export const PI = 3.14159;

export function sumar(a, b) {
  return a + b;
}

export class Calculadora { ... }
```

Ahora sólo queda **importarlos** desde el otro fichero:

```
import { PI, sumar } from './mathUtils.js';

console.log(PI);           // 3.14159
console.log(sumar(10, 5)); // 15
```

En el caso de que un módulo tenga una **única cosa** para exportar, tenemos una Exportación por Defecto. Solo puede haber una exportación default por fichero.

Al importar, **NO se usan llaves {}** y se le puede asignar **cualquier nombre** en el fichero de destino.

Al exportar:

```
export default class UsuarioService {
  obtenerUsuarios() {
    return ["Iker", "Lucía", "Marta"];
  }
}
```

Y al importar (sin llaves):

```
import ServicioDeUsuarios from './UsuarioService.js';

const servicio = new ServicioDeUsuarios();
console.log(servicio.obtenerUsuarios());
```

Buenas prácticas

- Nombres descriptivos para funciones y variables.
- Exporta solo lo necesario.
- Agrupa funciones relacionadas en módulos específicos.
- Evita dependencias circulares
- Documenta tu código.

Ejercicio 1 – XXX

Realiza una página web que tenga botones para permitir modificar las siguientes propiedades de una ventana: